

Nombre: Daniel Bernal Anzano

DNI: 73514866N

Asignatura: Inteligencia Artificial y Computación en la Nube **Fecha:** 25/09/2026

Instrucciones:

Se valorará el orden y limpieza. Una respuesta incomprensible se considerará no respondida. Es obligatorio respetar el formato de entrega. **Todos** los recursos **deben** ser entregados. El incumplimiento supondrá una calificación de cero.

Procura ser breve y responder a todo lo que se pide.

Explicar (brevemente) significa 2-3 líneas. Indicar (brevemente) significa 1-2 líneas. Nombrar significa 1 palabra. La respuesta **solo debe** añadirse después de cada pregunta; en otro lugar supondrá una calificación de cero.

Cualquier tipo de fraude o copia supondrá una calificación de cero.

Lee antes de empezar y analiza los recursos proporcionados. Organiza el tiempo.

Formato de entrega:

Una única carpeta, que será comprimida y entregada en un archivo .zip.

Este documento resuelto por el alumno, en la raíz de dicha carpeta, en .pdf y .docx.

Si es necesario, dentro de la carpeta principal habrá una o varias subcarpetas que incluirán **TODOS los recursos adicionales pedidos, desarrollados y usados.** Los recursos no entregados supondrán una calificación de cero en el ejercicio donde se pidan.

Las capturas deben incluir el nombre del autor y la palabra P1 (ej: Juan Perez P1). Los recursos se deben organizar en subcarpetas por ejercicios.

Formato mínimo:

dawm2d_opia_p1.zip

|.....dawm2d_opia_p1.pdf (tarea, en pdf)

|.....dawm2d_opia_p1.docx (tarea, en docx)

|.....recursos/ (carpeta para recursos adicionales, pedidos, desarrollados y usados)

|.....ejercicio_n/ (subcarpeta para recursos de ejercicio n)

Instrucciones.

La práctica se realizará en parejas y deberá incluir el nombre de ambos integrantes.

Evaluación.

Todos los apartados (CE) se ponderan por igual en la nota del RA.

PRÁCTICA 1 - Procesamiento de datos y modelo simple de IA [RA1] v2

Puedes realizar esta práctica en local (con Pycharm y Python en tu propio equipo) o en una plataforma en la nube como Google Colab.

Puedes usar esta [plantilla](#) como ayuda.

1) Principios fundamentales de la IA [RA1.A]

En 2-3 líneas, diferencia IA / Machine Learning. Clasifica el problema de predicción de precio en casas de california ([california housing price](#)) que vamos a resolver (indica si supervisado o no supervisado, indica si regresión o clasificación).

Entregable (memoria): un párrafo con la clasificación del problema justificada.

Respuesta: La Inteligencia Artificial crea sistemas capaces de realizar tareas que requieren comportamiento inteligente; el Machine Learning es una parte de la IA que aprende patrones a partir de datos. Es un problema supervisado porque tenemos características y una variable objetivo conocida, y es de regresión porque el precio de la vivienda es un valor numérico continuo.

2) Configuración del entorno [RA1.B]

Prepara un entorno de Python aislado para el proyecto e instala las librerías necesarias para procesar datos (numpy, pandas), entrenar modelos (scikit-learn) y graficar (matplotlib). Genera después un fichero con las dependencias instaladas.

Entregable (memoria, recurso): el fichero de dependencias generado (`requirements.txt`).

Respuesta: Se creó un entorno aislado con `python3 -m venv` y se instalaron numpy 2.5.3, pandas 3.0.6, scikit-learn 1.9.1, matplotlib 3.11.2 y joblib 1.6.0. El fichero generado con `pip freeze` se entrega en `recursos/requirements.txt`.

3) Técnicas de procesamiento de datos [RA1.C]

Realiza un análisis exploratorio básico del dataset: comprueba si hay valores nulos. Trata los valores nulos, descartando (o rellenando) los datos.

Entregable (recurso): el script con la comprobación (recurso). Una captura de la salida.

Respuesta: El dataset California Housing tiene 20.640 filas y ninguna de sus columnas contiene valores nulos. Se comprueba con `df.isna().sum()` y se trata con `dropna()`; en este caso se conservan las 20.640 filas. Todo el código está en `recursos/model_training.py`.

```
>>> from model_training import validate_data
>>> validate_data()
Daniel Bernal Anzano | P1
Valores nulos por columna:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
MedHouseVal 0
Filas antes: 20640
Filas después: 20640
Filas descartadas: 0
>>>
```

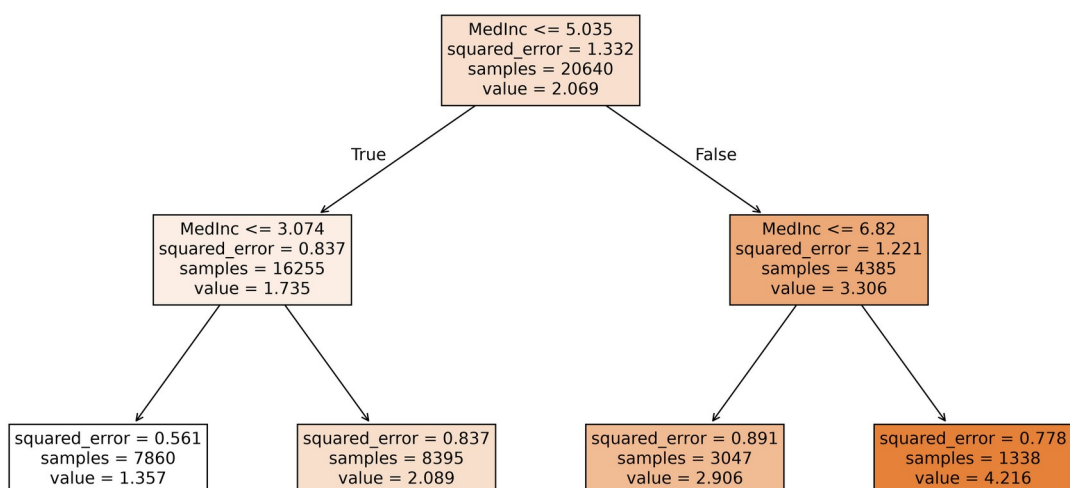
4) Estructuras de datos dinámicas, árboles y/o grafos [RA1.D]

Entrena un árbol de decisión pequeño sobre el dataset, con un `max_depth` bajo. Visualízalo para entender su estructura interna como árbol (nodos de decisión y hojas). Puedes dibujarlo con `plot_tree` de scikit-learn o con `graphviz`.

Entregable (memoria): una captura del árbol dibujado (con `plot_tree` o `graphviz`) junto con una frase explicando qué representa una hoja en ese árbol (memoria). Adjunta también la imagen png creada (memoria, recurso).

Respuesta: Se entrenó un `DecisionTreeRegressor` con `max_depth=2` y `random_state=42`, y se dibujó con `plot_tree` sin truncar, de modo que la imagen muestra el árbol entero hasta sus hojas. Una hoja es un nodo terminal que ya no hace más divisiones y contiene la predicción media del precio de las observaciones que llegan hasta ella. La imagen es `recursos/arbol_decision.png`.

Árbol de decisión (`max_depth=2`) - Daniel Bernal Anzano | P1



5) Estructuras de control y funciones [RA1.E]

Completa una función `plot_decision_tree(X, y, max_depth=None)` que dibuje un árbol de decisión. La función debe validar el parámetro `max_depth` (si no se indica o supera un límite razonable, debe usar un valor por defecto) antes de entrenar el árbol.

Entregable (recurso): un script con la función definida.

Respuesta: En la plantilla completada la función es `plot_decision_tree(X, y, max_depth=None)`: valida el parámetro con `validate_max_depth()` y, si no se indica, no es un entero válido o supera el límite razonable (`MAX_PLOT_DEPTH_LIMIT = 4`), usa el valor por defecto `DEFAULT_PLOT_DEPTH = 2`. Luego entrena su propio `DecisionTreeRegressor` con esa profundidad y lo dibuja con `plot_tree`. El modelo del apartado 7 se entrena aparte en `build_model()` con `MAX_DEPTH = 5`. El script se entrega en `recursos/model_training.py`.

6) Librerías y frameworks de ML [RA1.F]

Compara **dos** algoritmos de regresión de scikit-learn adecuados para este problema (por ejemplo, uno lineal y uno basado en árboles) en cuanto a velocidad, calidad predictiva e interpretabilidad.

Entregable (memoria): una tabla comparativa de 3 filas (velocidad, calidad, interpretabilidad) con tu conclusión sobre cuál usarás y por qué.

	Regresión lineal	Árbol de decisión
Velocidad	Muy rápida: 0,0167 s.	Rápida: 0,0449 s.
Calidad	MAE 0,5332; R ² 0,5758.	MAE 0,5223; R ² 0,5997.
Interpretabilidad	Alta mediante los coeficientes.	Alta mediante nodos, reglas y hojas.

Respuesta: Usaré el árbol de decisión porque obtiene un MAE y un R² mejores que la regresión lineal, sigue siendo muy rápido y permite visualizar e interpretar claramente sus decisiones. Además, es el modelo que se puede guardar, dibujar y analizar con las herramientas vistas en la UD1.

7) Desarrollo y entrenamiento de un modelo simple [RA1.G]

Entrena el modelo elegido con los datos de entrenamiento, evalúa su error sobre los datos de prueba con una métrica de regresión habitual (por ejemplo, error absoluto medio, en `sklearn.metrics`) y guarda el modelo entrenado en disco con `joblib`.

Entregable (recurso): El **script** que genera ese modelo (recurso).

El fichero del modelo entrenado `modelo_california.pkl` (recurso).

El valor de error obtenido (memoria).

Respuesta: El modelo elegido es `DecisionTreeRegressor(max_depth=5, random_state=42)`. Se entrenó con 16.512 ejemplos y se evaluó con 4.128 ejemplos de prueba. El MAE obtenido es **0,5223** y el MSE es 0,5245. El script es `recursos/model_training.py` y el modelo `recursos/modelo_california.pkl`.